Technologies avancées du web

MEAN

-

Angular 21+ avec Node : CSR & SSR

Pierre Pompidor

Angular 21+ : architecture hybride CSR & SSR

Cette nouvelle version permet de faire générer des pages sur le serveur

- pour améliorer le référencement (SEO)
- pour utiliser le cache des serveurs HTTP

Angular 21+ : architecture hybride CSR & SSR

Cette nouvelle version permet de faire générer des pages sur le serveur

- pour améliorer le référencement (SEO)
- pour utiliser le cache des serveurs HTTP

Le fichier **src/server.ts**

- est le serveur **Node.js**
- utilisant par défaut le framework **Express**

Angular 21+ : architecture hybride CSR & SSR

Cette nouvelle version permet de faire générer des pages sur le serveur

- pour améliorer le référencement (SEO)
- pour utiliser le cache des serveurs HTTP

Le fichier **src/server.ts**

- est le serveur **Node.js**
- utilisant par défaut le framework **Express**

Les endpoints peuvent

- délivrer des ressources statiques
- délivrer du JSON à destination de composants du front (CSR)
- délivrer des pages générées côté serveur (SSR)

Hiérarchie du projet Angular

Fusion front et back

```
projet
    package.json
    node_modules
    src
        index.html, styles.css
        server.ts   <= le code Node / Express
        ...
        app
            app.ts, app.html, app.css
            app.routes.ts
            ...
            Composant1  <= composant (front ou back)
              ...

    dist
        server      <= le code serveur pour Node
        browser     <= ressources statiques et pages SSR
```

Vue de l'application d'ecommerce

src/index.html :
<app-root></app-root>

src/app/app.html :

```
<app-homepage></homepage>
<nav>
  <button routerLink="/products">Products</button>
  <button routerLink="/cart">Cart</button>
</nav>
<router-outlet />
```

src/app/homepage/homepage.[ts,html,css] :
Composant SSR

src/app/products/products.[ts,html,css] :
Composant CSR
Appel de */api/products* via un service

src/app/cart/cart.[ts,html,css] :
Composant CSR

Table de routage (src/app/routes.ts)

Pour les composants hydratés côté front (CSR)

```
import { Routes } from '@angular/router';
import { Products } from './products/products';
import { Cart } from './cart/cart';

export const routes: Routes = [
  { path: 'products', component: Products },
  { path: 'cart', component: Cart },
];
```

ecommerce/src/app/server.ts

Endpoint de renvoi des fichiers statiques

```
server.use(express.static(browserDistFolder,
  {maxAge: '1y', index: false,}));
```

ecommerce/src/app/server.ts

Endpoint de renvoi des fichiers statiques

```
server.use(express.static(browserDistFolder,
  {maxAge: '1y', index: false,}));
```

Le booléen false permet l'invocation de routes par des composant du front

ecommerce/src/app/server.ts

Endpoint de renvoi des fichiers statiques

```
server.use(express.static(browserDistFolder,
  {maxAge: '1y', index: false,}));
```

Le booléen false permet l'invocation de routes par des composant du front

Exemple (en dur) d'un endpoint qui renvoit du JSON consommé par un composant chargé dans le navigateur

```
server.get('/api/homepage', (_, res) => {
  res.json([
      {"rayon":"Telephonie", "promotion":0},
      {"rayon":"Informatique", "promotion":15}
    ]);
  });
```

ecommerce/src/app/server.ts

Endpoint de gestions de toutes les routes SSR

```
server.get('{*splat}', async (req, res, next) => {
  const {protocol, originalUrl, baseUrl, headers} = req;
  commonEngine
    .render({
      bootstrap,
      documentFilePath: indexHtml,
      url: originalUrl,
      publicPath: browserDistFolder,
      ...
    })
    .then((html) => res.send(html))
    .catch((err) => next(err));
  })
```

ecommerce/src/app/server.ts

Endpoint de gestions de toutes les routes SSR

```
server.get('{*splat}', async (req, res, next) => {
  const {protocol, originalUrl, baseUrl, headers} = req;
  commonEngine
    .render({
      bootstrap,
      documentFilePath: indexHtml,
      url: originalUrl,
      publicPath: browserDistFolder,
      ...
    })
    .then((html) => res.send(html))
    .catch((err) => next(err));
  })
```

splat permet de récupérer la route

Les signaux

Les signaux servent à gérer un état **réactif**
en mettant automatiquement à jour les valeurs et les vues
qui en dépendent lorsqu'une donnée change.

Les signaux

Les signaux servent à gérer un état **réactif**
en mettant automatiquement à jour les valeurs et les vues qui en dépendent lorsqu'une donnée change.

Création par la fonction *signal()*

```
const count = signal(0);
console.log('The count is: ' + count());
```

Quelques méthodes :

```
const count: WritableSignal<number> = signal(0);

count.set(10);

count.update((value) => value + 1);

const doubleCount:Signal<number> = computed(() => count()*2);
```

Un exemple plus complet sur les signaux :

Un service gérant un panier d'articles

```
export class CartService {
  items = signal<CartItem[]>([]);

  total = computed(() =>
    this.items().reduce((sum, item)
      => sum + item.price * item.quantity, 0)
  );

  itemCount = computed(() =>
    this.items().reduce((sum,item) => sum+item.quantity, 0)
  );

  addItem(product:<CartItem>) {
    this.items.update(current => {
      ... // cas de l'existence
      return [...current, { ...product, quantity: 1 }];
    });
  }
```

Un exemple plus complet sur les signaux (suite) :

Un composant affichant en pied de page les infos du panier :

```
....
import { CartService } from './cart.service';

@Component({
  selector: 'app-cart-summary',
  standalone: true,
  template: `
    ...
    <footer>
      {{ cart.itemCount() }} articles
      pour un total de {{ cart.total() }}
    </footer>`
})
export class Homepage {
  cart = inject(CartService);
  ...
}
```